Automatisierung eines Fußballspiels

Die Entwicklung eines autonomen Fußballroboters
zur Teilnahme am RoboCup Junior 2026

Julius Kimpfler,
Jan Müller,
Dennis Skurka
Felix Wiedermann

Projektzeitraum
2024/2025

Inhaltsverzeichnis

1. Vorstellung 4
   1.1 Das Team 4
   1.2 Das Projekt 4
2. Hardware 5
   2.1 Fahrgestell und Gehäuse 5
   2.2 Aktoren 6
      2.2.1 Bewegungseinheit 6
      2.2.2 „Dribbler“ und „Schuss“ 7
   2.3 Sensoren 7
      2.3.1 Ballerkennung 7
      2.3.2 Linienerkennung 8
      2.3.3 Kamera 8
      2.3.4 Orientierungssensor 8
      2.3.5 Lichtschranke 9
      2.3.6 Entfernung und Position 9
   2.4 Platinen und Controller 9
      2.4.1 Bussysteme 10
      2.4.2 Powermanagement 10
      2.4.3 Verwendung von Reverse-Bauteilen 10
      2.5 Kommunikation 11
3. Software 11
   3.1 Mathematische Grundlagen 11
      3.1.1 Bewegungskontrolle 11
      3.1.2 Balllokalisierung 12
      3.1.3 Positionsbestimmung 12
      3.1.4 Linienerkennung 12
      3.1.5 Reflektionen 13
      3.1.6 Ballanfahrt 14
      3.1.7 Regelungstechnik / PID 14
   3.2 Prozessoren 14
      3.2.1 Main Prozessor (Teensy 4.0) 15
      3.2.2 Kamera (STM32H743) 15
      3.2.3 Ball Sensorik (STM32G47) 15

3.3 Taktiken 16
3.3.1 Abwehr 16
3.3.2 Angriff 16
3.3.3 Anspiel 17
3.3.4 Einzelspieler 18
4. Debuggen und Tools 18
4.1 Erfolgreiche Daten Erfassung 18
4.2 Bluetooth & Network Debugger 19
4.3 Hardware-Interface 19
5. Simulation 20
6. Technische Herausforderungen 20
6.1 Entwicklung und Design der Platinen 20
6.2 Entwicklung unter begrenztem Bauraum 21
7. Quellen 21

1. Vorstellung

1.1 Das Team

Unser Team auf der Europameisterschaft 2025 in Bari, Italien. Hierbei wurde mit diesem Projekt der 3.Platz belegt.

Wir sind vier Schüler aus der Region Friedrichshafen, besuchen derzeit die 11. Klasse und sind Mitglieder der Roboter-AG unserer Schule in Markdorf. Robotik ist ein fester Bestandteil unseres Alltags geworden, da wir diesem Thema nicht nur während der Schulzeit, sondern auch an Wochenenden und in den Ferien unsere Zeit widmen. Wir sind seit 2022 im selben Team unserer Roboter-AG. Seit eineinhalb Jahren arbeiten wir an unserem Projekt, einem selbst entwickelten autonomen Roboter, der in der Lage ist, Fußball zu spielen. Mit unserem Projekt bzw. einer vorherigen Arbeit in einem ähnlichen Themenbereich haben wir nun schon zweimal an der Europameisterschaft des RoboCup Junior, mit dem Teamnamen „**Bodenseekoalas**“, teilgenommen.

Wir haben in den letzten anderthalb Jahren viel gelernt, sowohl fachlich als auch in der Zusammenarbeit als Team. Wir wissen, wie wichtig ein guter Teamgeist ist und unterstützen uns gegenseitig mit unseren individuellen Stärken.

Innerhalb unseres Teams haben wir die verschiedenen Themenbereiche aufgeteilt. Jan und Felix sind vor allem für das Design der Mechanik zuständig. Julius und Jan kümmern sich um die Entwicklung der Elektronik und Dennis und Julius sind zuständig für das Entwickeln der Software. Felix entwickelt und arbeitet an den Taktiken.

Einmal in der Woche treffen wir uns am Freitagnachmittag in den Räumen unserer Schule, um gemeinsam an unserem Projekt zu basteln, doch auch außerhalb dieser Zeit, z. B. in Freistunden oder den Ferien, treffen wir uns, um unser Projekt weiterzubringen. So verbringen wir viel Zeit miteinander. Innerhalb unseres Teams haben wir einen sehr starken Teamgeist entwickelt. Dieser wird vor allem in stressigen Situationen auf den Wettbewerben auf die Probe gestellt. Doch genau diese Situationen schweißen uns als Team noch viel mehr zusammen und helfen uns, unser Projekt weiter voranzutreiben und so immer weiter dazuzulernen.

1.2 Das Projekt

Unser Projekt besteht aus zwei fast identischen Robotern, welche miteinander kommunizieren, um ein intelligentes und spannendes Spiel zu ermöglichen.

Unser V1 Roboter auf der German Open in Nürnberg Mai 2025

Im letzten Jahr (Juni 2024 bis August 2025) haben wir uns auf die Entwickelung eines einzelnen Roboters konzentriert, welcher auf einem Spielfeld mit Bande (also ohne Außenbereich) in der Lage war, mit einem aktiv sendenden Infrarot-Ball, Tore zu erzielen und sein eigenes Tor zu verteidigen. Dies war unsere erste Entwicklungsstufe (V1).

Im letzten halben Jahr haben wir unseren Roboter neu aufgebaut, um nun auf einem etwas größeren Feld mit einem Torraum und einem Außenbereich, welcher nicht befahren

werden darf, zu spielen. Wir haben nun auch anstatt eines Roboters, welcher gegen einen einzelnen anderen Roboter antritt, zwei Roboter, welche gegen zwei weitere autonome Roboter antreten, entwickelt. Dies eröffnet neue Möglichkeiten, wie etwa das Zusammenspiel von zwei Robotern mit bestimmten Taktiken, aber es führt auch zu neuen Herausforderungen, wie z. B. der erforderlichen Kommunikation zwischen den beiden Robotern (Entwicklungsstufe V2).

Eine weitere Neuerung ist zudem, dass wir nun einen kleineren 3D-gedruckten Ball verwenden, welcher ebenso aktiv Infrarot-Signale sendet. Dieser ist zudem nicht mehr transparent bzw. grau, sondern orange. Somit ist es uns möglich, den Ball auch mithilfe einer Kamera zu erkennen und zu lokalisieren.

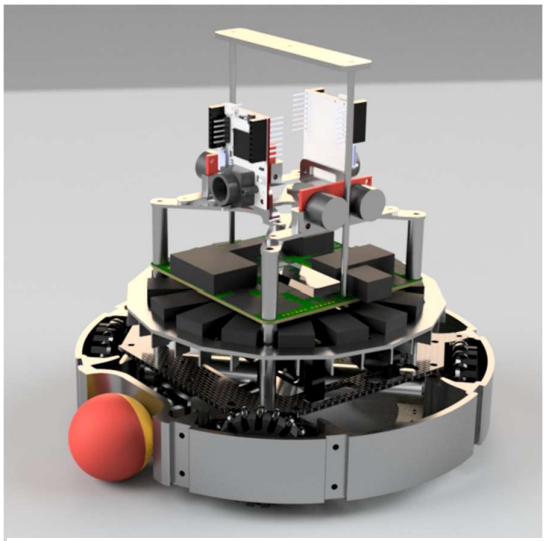
Die Entwicklung eines solchen Roboters ist für uns ein großer Lernprozess, in dem wir viel dazulernen. Die nötigen Grundlagen werden uns vor allem von den ehemaligen Teilnehmern unserer AG vermittelt, welche wir dann umsetzen und weiterentwickeln. Die ehemaligen Teilnehmer sind zwar oft schon mitten im Studium, dennoch kommen viele von ihnen regelmäßig zu uns nach Markdorf und helfen uns mit dem, was sie wissen. Hilfe und Wissen bekommen wir auch von den freiwilligen Helfern in unserer AG. Dies sind meist selbst Ingenieure, die Freude und Spaß daran haben, uns etwas beizubringen.

Sobald ein neues Team in unserer AG im Selbstbaubereich anfängt, bekommen wir jede Woche einen Vortrag von einem unserer Betreuer angeboten, in dem grundlegende Kenntnisse zu Elektronik, Software und Mechanik vermittelt werden, beispielsweise den Umgang mit CAD-Programmen oder verschiedene Programmiersprachen. In diesem Bereich gilt für uns auch ein hohes Maß an Eigenstudium, um uns in diese Bereiche einzuarbeiten.

2. Hardware

2.1 Fahrgestell und Gehäuse

Zum Bau unseres Chassis und des Grundgerüsts verwenden wir hauptsächlich das 3D-Druckverfahren. Uns steht an unserer Schule ein 3D-Drucker (Prusa MK3) zur Verfügung, welcher allerdings aufgrund der Druck-geschwindigkeit und der Materialkompatibilität in den meisten Fällen nicht zum Einsatz kommt. Stattdessen verwenden wir unsere beiden privaten Drucker (Creality Ender3 V3 KE und ein Bambulab A1), um die mechanischen Bauteile zu fertigen. Da oft auch Änderungen schnell und spontan zustande kommen, ist ein schnelles Prototyping mit dem 3D-Drucker unerlässlich.



Das aktuelle V2 Design im 3D Modell
(Platinen in vereinfachter Darstellung)

Oft sind jedoch die Eigenschaften des Materials ein beschränkender Faktor. Wir nutzen aus diesem Grund spezielle Materialien, wie ABS, oft auch mit Faserverstärkung (z. B. mit Glasfasern oder Carbonfasern), PETG oder Nylon (PA6 / PA12).

In bestimmten Fällen können wir allerdings aus Gewichtsgründen keinen 3D-Druck verwenden. In diesen Fällen fräsen wir beispielsweise aus Carbon- oder Glasfaserplatten äußerst stabile und gleichzeitig sehr leichte Bauteile. Hierfür nutzen wir eine CNC-Fräse. Bei der Arbeit mit Werkstoffen wie Carbon, welche unter Umständen bei falscher Handhabung gesundheitsgefährdend sind (durch z. B. Fasern / Staub), achten wir besonders darauf, keine Gefahr einzugehen. Zudem kommt erschwerend hinzu, dass Carbon ein sehr schwieriges Material zum Fräsen ist, da ein zu stumpfer Fräskopf schnell zur Verklumpung der Spindel und damit zum Absplittern des Fräskopfes von der Fräse führen kann.

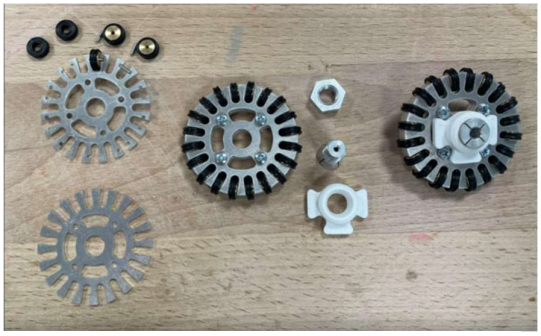
Aus diesen Gründen testen wir jedes Modell, welches wir aus Carbon fräsen wollen, erst einmal aus einem einfachen Werkstoff, wie z. B. Holz, um Designfehler auszuschließen. Dies spart zudem Kosten, da sowohl Carbon als auch Glasfaser sehr teure Rohstoffe sind.

Um die Reparatur und Weiterentwicklung der Roboter so einfach wie möglich zu halten, verwenden wir nur Gewindeschrauben mit dem gleichen Antrieb. Nach mehreren Tests haben wir uns für den Torx Antrieb der Größe 10 (= TX10) entschieden. Dieser Antrieb überzeugt uns vor allem in seiner Haltbarkeit und seiner hervorragenden Kraftübertragung. Kombiniert mit unseren M3 Einschmelzmuttern lassen sich 3D gedruckte Teile unkompliziert mit weiteren Komponenten oder Platinen verbinden. Die Einbringung der Einschmelzmuttern ist mit einem alten Lötkolben und einer entsprechenden Aussparung im 3D-Modell einfach umzusetzen.

2.2 Aktoren

2.2.1 Bewegungseinheit

Die Roboter (V2) sind mit jeweils vier Motoren zur Bewegung des Roboters auf dem Spielfeld ausgestattet. Bei unserem ersten Roboter (V1) verwendeten wir nur drei Motoren, um den Einstieg zu erleichtern. Wir haben uns dazu entschieden, dass die zusätzliche Leistung eines vierten Motors in unserem neuen Robotersystem notwendig für eine erfolgreiche und schnelle Steuerung der inzwischen etwas schwereren Roboter ist.
Die Roboter werden allerdings nicht mit Hilfe normaler Räder bewegt, sondern mithilfe sogenannter Omniwheels. Diese ermöglichen es dem Roboter, in jede beliebige Richtung zu fahren, ohne sich um die eigene Achse zu drehen. Omniwheels funktionieren sowohl bei drei Antriebsmotoren als auch bei vier Antriebsmotoren . Wir haben uns dazu entschieden, 6-V-Motoren von Faulhaber zu verwenden, um eine möglichst hohe Leistung bei sehr geringem Gewicht zu erreichen, obwohl unser Roboter ein 12-Volt-System verwendet. Die Omniwheels sind von der Firma Homburger in Markdorf gefertigt, da wir auf dem freien Markt keine geeigneten Räder in einer solchen Ausführung finden konnten.



Eines der von Homburger gefrästen Omniwheels beim Zusammenbau.

2.2.2 ‚Dribbler' und ‚Schuss'

Die Roboter sind in der Lage, mithilfe eines sogenannten ‚Dribbler'-Mechanismus, den Ball anzudrehen, so dass dieser ständig in Richtung des Roboters gedrückt wird und somit eine gute Ballkontrolle möglich ist. Solch einen Mechanismus haben wir in unserer V1-Version mithilfe eines weiteren 6-V-Motors realisiert. Wir hatten jedoch Schwierigkeiten, diesen bisher in unserer V2-Version einzufügen.
Dies war vor allem dem geschuldet, dass nun der Bereich zwischen den zwei vorderen Motoren (V1: 120° / V2: 90°) aufgrund des zusätzlichen Motors geringer ist und somit kein Mechanismus hineinpasste.

Der neueste Ansatz ist jedoch, anstatt eines länglichen Brushed-Motors, einen flachen Brushless-Drohnen-Motor zu verwenden. Dies ist allerdings noch nicht abschließend getestet.
Anstelle eines solchen Dribbler-Mechanismus haben wir nun eine sogenannte ‚Schuss'-vorrichtung eingebaut. Diese soll dabei helfen, den nicht befahrbaren Torraum zu überwinden. Der ‚Schuss' soll den Ball möglichst kräftig vom Roboter in eine bestimmte Richtung abstoßen. Hierfür nutzen wir ein sogenanntes ‚Solenoid', also eine Spule, welche einen Eisenstift anzieht und somit den Ball wegschießt. Wir verwenden hierfür eine selbstgewickelte Spule, um eine möglichst hohe Schusskraft zu erreichen.

2.3 Sensoren

2.3.1 Ballerkennung

Zur Erkennung, in welchem Abstand und in welchem Winkel zum Roboter der Ball sich befindet, nutzen wir mehrere selbst entwickelte Infrarotsensoren. Der Ball sendet Infrarotstrahlen mit einer Pulsfrequenz von ca. 40 kHz und einer Wellenlänge von 940 nm aus.

Da die auf dem Markt verfügbaren Infrarotsensoren lediglich eine Modulationsfrequenz von 38 kHz verwenden, haben wir uns dazu entschieden, eigene angepasste Sensorplatinen, welche die von mehreren Infrarotdioden empfangenen Frequenzen filtern und mehrfach verstärken, zu verwenden.

Insgesamt haben wir auf unserem aktuellen Roboter 16 solcher Sensoren kreisförmig angeordnet, um jeweils die Stärke des einfallenden Signals zu bestimmen und so die Entfernung und den Winkel des Balls zum Roboter zu bestimmen. Zusätzlich zu den 16 eigenen Sensoren, welche den Ball bis auf wenige Zentimeter genau im Umkreis von ca. zwei Metern lokalisieren können, verwenden wir noch acht gekaufte 38-kHz-Sensoren, um den Ballwinkel bei größerer Entfernung grob bestimmen zu können.
Die Ballerkennung und Berechnung wird zudem von einem eigenen Controller durchgeführt (einem STM32G47, siehe Kapitel 3.2.2).

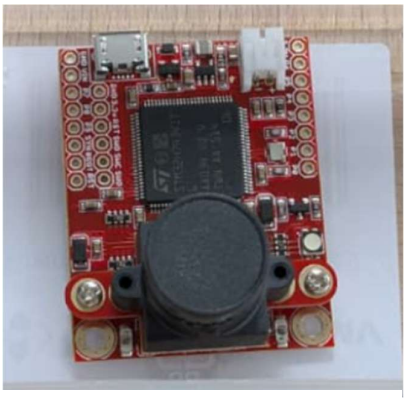
2.3.2 Linienerkennung

Da es im Falle unseres V2-Roboters einen Bereich des Spielfelds gibt, der nicht befahren werden darf, ist es notwendig, dass unsere beiden Roboter mit einer Sensorik zum Erkennen der weißen Außenlinie ausgestattet sind. Wir verwenden hierbei insgesamt 40 rote LEDs, welche auf den Boden direkt unter dem Roboter strahlen und in Kombination mit 40 Phototransistoren, welche exakt neben jeder LED positioniert sind, es möglich machen, durch die Stärke der Reflexion bzw. Absorption des roten Lichts zu erkennen, ob der Untergrund grün oder weiß ist.
Ein weiteres Problem beim Vermeiden der Linien ist die Geschwindigkeit, mit welcher der Roboter sich bewegt. Die maximale Geschwindigkeit auf 2 m Beschleunigungsweg beträgt 2 m/s. Um den Roboter also innerhalb von ca.10 cm abzubremsen, muss sowohl die Masse des Roboters sehr gering sein als auch die Elektronik und Software sehr schnell reagieren.

Die voll funktionsfähige Linien-Platine bei ihrem ersten Einsatz.

2.3.3 Kamera

In unserer V1-Version haben wir eine PixyCam 2.1 verwendet, um das farblich markierte Tor (gelb bzw. blau) eindeutig zu erkennen. Die PixyCam ist allerdings sehr langsam und empfindlich gegenüber den den Lichtverhältnissen auf dem Feld und somit eigentlich ungeeignet für unsere Anwendung.
Für unseren V2-Roboter verwenden wir zwei OpenMV H7 R2, welche einen eigenen, mit MicroPython programmierbaren, Prozessor verbaut haben. Dieser kümmert sich um grundlegende Kalkulationen der Bilderkennung. Mit dieser Kamera hatten wir im vergangenen halben Jahr jedoch große Verbindungsprobleme. Diese waren schließlich auf einen Fehler in der Verkabelung zurückzuführen und konnten somit von uns gelöst werden.


Eine der zwei verwendeten OpenMVH7 R2 Kameras mit STM32 Chip und Originallinse.

Die OpenMV H7 R2 besitzt ein M12 Gewinde, um die verwendete Linse zu tauschen. Die mitgelieferte Linse hat nur einen geringen Sichtwinkel, weshalb wir auf eine Linse mit größerem Sichtfeld wechselten, um damit eine bessere Übersicht über das Spielfeld zu ermöglichen.

2.3.4 Orientierungssensor

Zur Orientierung des Roboters nutzen wir einen BNO055 bzw. BNO085. Dies ist ein weit verbreiteter Bosch-9-Achsen-Orientierungssensor, welcher sehr zuverlässig die Drehung um die verschiedenen Achsen bestimmt. Somit ist es möglich, dass die Roboter sich immer zum gegnerischen Tor ausrichten, um z. B. Eigentore zu vermeiden.

2.3.5 Lichtschranke

Um festzustellen, ob sich der Ball aktuell unter der Kontrolle des jeweiligen Roboters befindet, verwenden wir eine Lichtschranke, welche mithilfe einer kleinen Laserdiode und einem LDR feststellt, ob sich ein Gegenstand in der Ballmulde am Roboter befindet.

Zu Beginn verwendeten wir als Lichtquelle eine handelsübliche THT LED. Schnell merkten wir jedoch, dass das ausgegebene Licht sehr schwach war und schon bei einer leichten Erhöhung der Umgebungshelligkeit für den LDR nicht mehr vom Umgebungslicht zu unterscheiden war. Um dem entgegenzuwirken, nutzten wir Laserdioden und verbesserten so die Funktion unserer Lichtschranke.

2.3.6 Entfernung und Position

Die Positionierung in X- bzw. Y-Richtung auf dem Feld ist einer der schwierigsten Bereiche unserer Sensorik, da wir aufgrund von möglichen Störungen des Gegners keine lichtemittierenden Entfernungs- bzw. Positionierungssensoren, wie Lidar- oder TOF-Sensoren, verwenden dürfen, da diese zu nahe am Spektrum des Infrarotballs liegen würden.
Um dies zu umgehen, verwenden wir ausschließlich Ultraschallsensoren oder Sensoren, welche in Computermäusen verwendet werden. Beide Varianten sind allerdings sehr ungenau und deshalb wenig erfolgreich. Daher errechnen wir die Position des Roboters vor allem durch zwei Kameras.

2.4 Platinen und Controller

Die gesamte Elektronik wird von uns selbst designt und umgesetzt. Wir nutzen „Eagle“ von Autodesk, um all unsere Platinen zu entwerfen und lassen diese anschließend von JLCPCB fertigen.

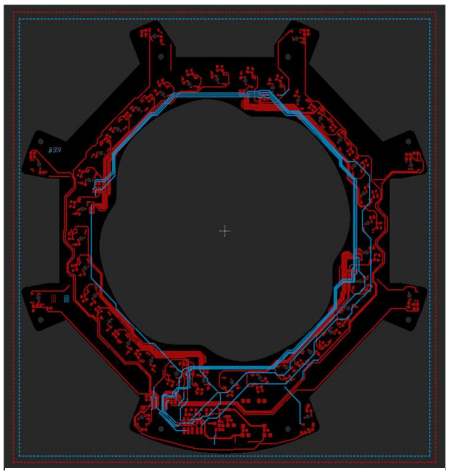
Glücklicherweise haben wir an unserer Schule Zugang zu einem Reflow-Ofen zum Bestücken der Platinen. Wir nutzen inzwischen Bauteile bis zur Formgröße 0603, um möglichst platz- und gewichtseffizient auf unserem Roboter zu arbeiten. Die gesamten Schaltungen wurden von uns entworfen und entwickelt. Dank gilt hier unseren Betreuern, welche die Platinen regelmäßig überprüft und auf Fehler kontrolliert haben. Insgesamt verwenden wir auf unserem Roboter bis zu 21 Platinen:

- 1x Linienerkennung mit eigenem DC-DC-Wandler zur Versorgung der 40 Power-LEDs
- 1x Mainboard mit Powerlayout und Maincontroller
- 1x „IR-Ring“ mit Co-Prozessor für die Ballberechnung
- 16x Infrarotsensorplatinen (Eigenbau)
- 1x Interface-Platine mit eigenen I2C-Portexpandern zur einfachen Anbindung
- Brushless-ESC für ‚Dribbler‘-Motor (Eigenbau / gerade in Entwicklung)

Diese zu entwickeln, zu bestücken und zu testen ist ein anstrengender und langwieriger Prozess der mehrere Monate dauert.

2.4.1 Bussysteme

Eine weitere Herausforderung ist die zuverlässige Kommunikation über ein SPI-Bussystem, auch platinenübergreifend. Diese verwenden wir vor allem aufgrund der Vorteile im Bereich der Geschwindigkeit zur Datenübertragung gegenüber von I2C-Bussystemen, welche nur im Bereich des Benutzer-Interface wie Schaltern, Knöpfen oder Displays zum Einsatz kommen. Verbindungsprobleme mit dem Bussystem haben wir allerdings erfolgreich beseitigt, indem wir z.B. auf den Platinen soweit es geht, jegliche analoge Signale von digitalen Systemen trennen oder diese gegebenenfalls mit einer Masselage abschirmen.

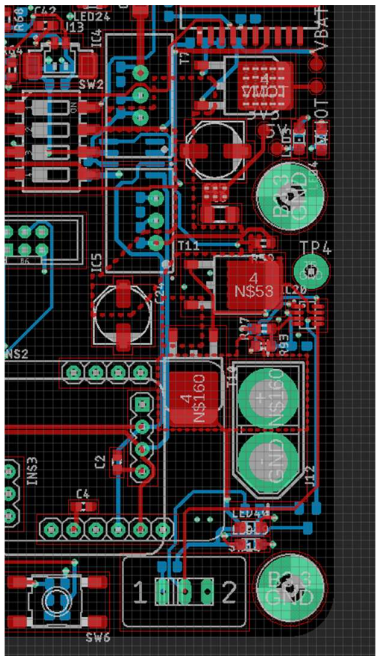


In blau sieht man die unterste Kupferschicht mit dem kreisförmig verlaufenden SPI-Bus. In rot sind alle analogen Versorgungen und Signale eingezeichnet.

So auch am Beispiel unserer Linienplatine. Hier ist deutlich zu erkennen, dass der SPI-Bus vollständig und isoliert auf der Platinenunterseite verläuft und somit durch die auf einer inneren Bahn verlaufenden Masseschicht abgeschirmt wird.

2.4.2 Powermanagement

Da wir zeitweise mehr als 1 Ampere pro Antriebsmotor benötigen, ist ein ausgereiftes Powerlayout bei den Platinen unerlässlich. Die Platinen sind dafür ausgelegt, mehr als 20 Ampere kurzzeitig auszuhalten, um genug Leistung zum Betrieb des ‚Schuss'-Mechanismus zu bieten. Die gesamte Stromversorgung wird von einem Power-MOSFET geschaltet, um möglichst kleine mechanische Schalter zu verwenden. Da die Leiterbahnen sehr dünn sind, müssen wir in den Hochstrombereichen sogenannte Polygone verwenden, um eine größere Kupferfläche als Leiter zu schaffen. Zusätzlich kann der gesamte 12-V-Bereich der Platine über einen weiteren FET abgetrennt werden, z. B. zum Testen der Software ohne Motoren. Auf unserer Platine befinden sich zudem zwei DC-DC-Wandler, um 5 V und 3,3 V bis 1 bzw. 2 Ampere bereitzustellen. Zur Regelung der Motoren sind vier bzw. fünf Vollbrücken-ICs verbaut, welche mit einer thermischen Kopplung zur Platine auch größere Motoren regeln können.



Ein Teil unseres Powerlayouts auf der V5.0 Version unserer Hauptplatine. Mittig erkennt man sowohl den zur Stromversorgung geplanten XT60 Stecker als auch mehrere Power MOSFETs.

2.4.3 Verwendung von Reverse-Bauteilen

Aufgrund der negativen Erfahrungen anderer Teams und Teilnehmenden der letzten Jahre des RoboCups, haben wir uns dazu entschieden, sogenannte Reverse-Bauteile auf unserer Liniensensorplatine einzusetzen. Reverse-Bauteile (in diesem Fall Reverse-LEDs und Reverse-PTs) sind Bauteile, die auf der gegenüberliegenden Platinenseite verbaut werden und durch gezielt platzierte Aussparungen in der Platine durch diese hindurch agieren (siehe Abb.). Das hat den Vorteil, dass es somit möglich ist, die Unterseite des Roboters vollständig bauteilfrei zu halten. In den letzten Jahren kam es vermehrt zu

Problemen, da Bauteile abgerissen oder kurzgeschlossen wurden, welche offen und ungeschützt dem Spielfeldboden ausgesetzt wurden. Dies hoffen wir, durch unseren neuen Ansatz zu umgehen.

2.5 Kommunikation

Da ein Großteil der Taktiken mit zwei Robotern und eines intelligenten Spiels nur mit einer schnellen und beidseitigen Kommunikation möglich ist, verwenden wir pro Roboter ein HC05-Bluetooth-Modul, welches, sobald es per AT-Protokoll konfiguriert wurde, eine automatische Verbindung zum zweiten Roboter herstellt, um den beidseitigen Informationsaustausch per UART-Schnittstelle zu ermöglichen. Es werden grundlegende Informationen wie Ballentfernung und angenommene Position des Roboters auf dem Spielfeld ausgetauscht und somit festgelegt, welcher Roboter im Angriff oder in der Abwehr aktiv sein soll. Hierfür haben wir uns ein spezielles Verständigungsmuster überlegt (siehe Kapitel 3.3).

3. Software

3.1 Mathematische Grundlagen

Unsere mathematischen Formeln für die Software sind alle selbst herausgearbeitet oder recherchiert. Zum Entwerfen dieser Formeln benutzen wir Goodnotes und mehrere Python-Simulationen, mit denen wir z.B. verschiedene Techniken der sogenannten Ballanfahrt simulieren können.

Größen:

- W => Winkel angegeben in Grad (°) von -180 bis 180 / relativ zum Spielfeld
- D => Entfernung in Prozent (%) von 100 (nah) bis 0 (fern)
- S => Signalstärke in Prozent (%) von 0 bis 100
- P => Parameter als float
- V => Wert des Vektors
- F => Faktor / Multiplikator
- ω => Winkelgeschwindigkeit
- Z => Zeit in Millisekunden

3.1.1 Bewegungskontrolle

Die Verwendung von Omniwheels hat, wie schon erwähnt, den Vorteil, dass der Roboter sich in jede beliebige Richtung bewegen kann, ohne sich zu drehen. Eine Bewegungssteuerung mit Omniwheels ist jedoch deutlich schwieriger in der Kalkulation umzusetzen.

Die Errechnung von drei (Version 1) bzw. vier (Version 2) verschiedenen Motorgeschwindigkeiten ($\omega_{Motor[n=1-4]}$) pro Roboter aus einem gewünschten Fahrwinkel (W_{Fahr}), einer Bewegungsgeschwindigkeit (ω_{Motor_Max}) und einer Drehgeschwindigkeit um die eigene Achse (ω_{PID}) ist wie folgt möglich:

Wir nutzen folgenden Formelsatz für vier Motoren:

$\omega_{Motor[n]} = sin(W_{Fahr}+(90*n)-45) + \omega_{PID}$

Für 3 Motoren ergibt sich:

$\omega_{Motor[n]} = sin(W_{Fahr}+(120*n)-60) + \omega_{PID}$

Die Werte $\omega_{Motor[n]}$ werden dann so weit erweitert, bis einer der drei bzw. vier Werte = ω_{Motor_Max} ist.

Somit wird gewährleistet, dass immer die maximale Bewegungsgeschwindigkeit für die gegebene maximale Drehgeschwindigkeit der Motoren (ω_{Motor_Max}) erreicht wird.

3.1.2 Balllokalisierung

Um den Ballwinkel (W_{Ball}) aus den 16 einzelnen Signalstärken (SSensor [n=1-16]) zu errechnen, muss die X bzw. Y Entfernung des Roboters zum Ball errechnet werde:

$V_{X_Summe} = \sum_{n=0}^{n=31} (S_{Sensor[n]} * sin\ (n*11.25°))$

$V_{Y_Summe} = \sum_{n=0}^{n=31} (S_{Sensor[n]} * cos\ (n*11.25°))$

Aus diesen beiden Vektoren kann schließlich der Winkel (W_{Ball}) bzw. eine Entfernung (D_{Ball}) errechnet werden.

$W_{Ball} = tan^{-1}(V_{X_Summe}/V_{Y_Summe});$

Für die ungefähre Entfernung ergibt sich daraufhin:

$D_{Ball} = \sum_{n=0}^{n=31} (S_{Sensor[n]})$

Für die Richtung haben wir, wie in 2.3.1 angesprochen, mehrere Infrarot-Sensoren eingesetzt. Diese sind jeweils im Abstand von 22,5 ° angeordnet. Somit hat jeder Sensor seinen einzigartigen Winkel von 0° – 337,5°. Nun benutzen wir die Stärke des Signals jedes Sensors als Gewichtung jedes Winkels.

3.1.3 Positionsbestimmung

Für die Positionsbestimmung verwenden wir zwei Ultraschall-Sensoren, die uns die Distanz zu den seitlichen Wänden ausgeben. Zusätzlich wird die Position näherungsweise durch die Kamera bestimmt.

3.1.4 Linienerkennung

Die Linienerkennung besteht aus zwei Berechnungen. Einmal wird geprüft, ob eine Linie von einem der acht äußeren Sensoren erkannt wird. Sobald dies der Fall ist, wird der Roboter mit maximaler Leistung in einem fest definierten Winkel $W_{Sensor[n]}$ [1] weg von der Linie beschleunigt. Hierfür gilt:
$W_{Fahrt} = W_{Sensor[n]} * 45° – 315°$

Falls der innere Ring mit 32 Sensoren eine Linie erkennt, wird die Position der Linie bzw. der Winkel in dem der Roboter auf der Linie steht, wie folgt errechnet:

[1] [n] steht hier für die verschiedenen werte je nach Nummer n

$V_{X_Summe} = \sum_{n=0}^{n=31} (S_{Sensor[n]} * sin(n*11.25°))$

$V_{X_Summe} = \sum_{n=0}^{n=31} (S_{Sensor[n]} * sin(n*11.25°))$

$W_{Linie} = tan^{-1}(V_{X_Summe}/V_{Y_Summe})$

Danach wird der Roboter in die genau entgegengesetzte Richtung beschleunigt.

$W_{Fahrt} = W_{Linie} - 180°$

3.1.5 Reflektionen

Wir benutzen eine Reflektionsformel um herauszufinden, in welchem Winkel wir den Ball gegen die Bande / einen Körper schießen müssen, damit er ins Tor rollt. Das ist nützlich, falls der gegnerische Roboter zwischen unserem Roboter und dem Tor steht. Die Punkte S und Z beschreiben jeweils die Start- und Zielkoordinate.

Wenn $S=(x_S,y_S)$ und $Z=(x_Z,y_Z)$ dann gilt für den Abprallpunkt P auf der Wand: $P = (0, y_S + a)$

Wobei „a“ die y-Distanz vom Startpunkt zum Abprallpunkt darstellt.

Nun kann man mit b die Distanz vom Abprallpunkt zum Zielpunkt errechnen und es ergibt sich die Gleichung:

$$y_Z - y_S = a + b$$

Und somit:

$$b = \frac{a \cdot x_Z}{x_S}$$

Zusammengefasst ergeben sich folgende Gleichungen

$$b = y_Z - y_S - a \quad \text{und} \quad b = \frac{a \cdot x_Z}{x_S}$$

Nach Gleichstellung ergibt sich:

$$\frac{a \cdot x_Z}{x_S} = y_Z - y_S - a$$

$$a = y_Z - y_S - \frac{x_Z \cdot a}{x_S}$$

Nach einer Substitution der Komponenten, kann man die Gleichung weiter vereinfachen:

$$a = \frac{c}{d + 1}$$

Mit dieser Formel lässt sich die y-Koordinate des Punktes und somit auch der Winkel berechnen, um den Ball mit einem Schuss Richtung Bande anzusteuern:

$$P_y = y_S + \frac{y_Z - y_S}{\frac{x_Z}{x_S} + 1}$$

3.1.6 Ballanfahrt

Die Ballanfahrt wird in mehrere Schritte unterteilt. Der Versuch ist, immer aus jeder Position erst einmal auf eine Kreisbahn um den Ball herum zu gelangen. Diese Kreisbahn hat optimalerweise einen Abstand von ca. 15 cm bis 25 cm zum Roboter. Der optimale Bewegungswinkel (Wfahrt) um auf diese Kreisbahn zu gelangen, wird wie folgt berechnet: $W_{Fahrt} = W_{Ball} \pm \tan^{-1}(20 / D_{Ball})$

Um dann von dieser Kreisbahn in eine Position zu gelangen, in welcher der Roboter den Ball einfach in die Ballmulde aufnehmen kann, soll der Roboter auf dieser Kreisbahn um den Ball fahren, bis der Ball in einem Bereich von ±20° vor ihm liegt. Der dann benötigte Fahrtwinkel wird mit der Gleichung:

$W_{Fahrt} = W_{Ball} * 1.1$ berechnet.

3.1.7 Regelungstechnik / PID

Da unser Roboter über Omniwheels verfügt, ist die Drehung um die eigene Achse, also die Blickrichtung (W_{Blick}), unabhängig von der Fahrtrichtung (W_{Fahrt}).

Der Winkel W_{Blick} kann also dauerhaft frei gewählt werden. Dafür verwenden wir einen sogenannten PID Regler. Dieser ermöglicht es uns, den Roboter exakt auf jeden nötigen Winkel zu drehen (W_{Soll}). Diese Drehbewegung wird automatisch im Hintergrund mit in die Bewegungsbefehle eingerechnet. Der PID-Regler errechnet als erstes den sogenannten Error (W_{Error}) aus:

$W_{Errror} = W_{Blick} – W_{Soll}$.

Um nun die erforderliche Drehgeschwindikeit (ω_{PID}) zu errechnen, nutzt man folgende Gleichung:

$\omega_{PID} = (W_{Error} * P_K) + (W_{Summe_Error}) * P_I * Z_{Zyklus}) + ((W_{Error} - W_{Last_Error}) / Z_{Zyklus} * P_D);$

Hierbei ist P_K der Parameter für den Proportional-Anteil, P_I der Parameter für den Integralanteil und P_D der Parameter für den Differentialanteil.

Sobald diese drei Parameter richtig abgestimmt sind, ist es möglich, den Roboter ohne Oszillationen sehr schnell und präzise zu steuern. Die PID-Konfiguration muss immer dann angepasst werden, wenn sich die Massenverteilung oder Masse selbst auf dem Roboter verändert, da die Massenträgheit hier einen großen Einfluss hat.

3.2 Prozessoren

Wie schon im Kapitel 2.3.1 und 2.3.3 erwähnt, verwenden wir nicht nur einen lokalen Controller, sondern verteilen die verschiedenen Aufgaben auf insgesamt drei Prozessoren.

Alle drei Prozessoren sind miteinander über einen bei uns standardisierten Steckverbinder verbunden. Dieser überträgt sowohl Power als auch I2C oder SPI-Signale. Hierbei handelt es sich um einen 2,00 mm Wannenstecker mit Flachbandkabel, um eine möglichst übersichtliche und ordentliche Verkabelung zu gewährleisten.

3.2.1 Main Prozessor (Teensy 4.0)

Der Teensy 4.0 agiert auf unserer Main Platine als zentrale Steuereinheit, welche sich um das Abfragen von Sensorwerten, die Auswertung dieser und die anschließende Steuerung von Motoren übernimmt.

Wir verwenden hierfür einen Teensy 4.0, da dieser einer der leistungsstärksten Microcontroller auf dem Markt ist und gleichzeitig viele Interface- und Bussysteme verarbeiten kann. Er besitzt eine integrierte Recheneinheit, um trigonomische Berechnungen schnell und präzise umzusetzen.

Wir verwenden aufgrund der hohen Anzahl von benötigten Anschlüssen, sowohl die Standardpins als auch spezielle Zusatzpins auf der Controllerunterseite.

Durch aufwändige Zeitoptimierung des Programms erreichen wir inzwischen eine durchschnittliche Programmlaufzeit eines Rechenzyklus von ca. 1500 Microsekunden. Dies ist eine deutliche Verbesserung zu unseren ersten Versuchen von ca. 20 Millisekunden pro Rechenzyklus.

3.2.2 Kamera (STM32H743)

Auf dem Prozessor der Kamera verwenden wir, im Gegensatz zu den anderen beiden Prozessoren auf denen C++ verwendet wird, Micro Python als Programmiersprache.

Auf der Kamera errechnen wir die gesamte Bilderkennung, sowie grundlegende Werte zur Positionsbestimmung. Die Daten werden anschließend in einzelne Pakete verpackt und per UART-Schnittstelle zum Teensy versendet. Hierfür verwenden wir unsere eigene Codierung:

1) zuerst wird ein Indikation- Byte gesendet. In diesem Fall ein ‚%'- Zeichen, um dem Teensy zu signalisieren, dass die Daten nun gesendet werden.

2) danach folgen zwei Byte mit der Information über die Sichtbarkeit des Tores.

(**‚ff'** = weder gegnerisches Tor noch eigenes Tor sichtbar / **‚tf'** = eigenes Tor sichtbar, aber gegnerisches nicht / **‚ft'** = gegnerisches Tor sichtbar, aber eigenes Tor nicht / **‚tt'** = beide Tore sichtbar).

3) dann folgt ein Byte für die Positionierung des Tors auf dem Kamerabild

4) ein weiteres Byte zur geschätzten Nähe des Tores zur Kamera

5) es folgt ein Schluss Signal: ‚$'

3.2.3 Ball Sensorik (STM32G47)

Dieser Prozessor wird von uns auf unserer selbst entwickelten Platine, dem sogenannten „IR-Ring" (siehe Kapitel 2.4), verwendet. Der Chip kommt zum Einsatz, um als intelligenter ADC zu fungieren. Er wandelt also die analogen Eingangswerte der insgesamt 24 IR-Sensoren in digital zugängliche Informationen um, welche sowohl per i2C, als auch per SPI-Schnittstelle ausgelesen werden können. Zusätzlich berechnet dieser aus den gemessenen Werten den genauen Ballwinkel und die Ballentfernung und agiert zudem als digitaler Portexpander um z.B. Debugging LEDs zu steuern.

3.3 Taktiken

3.3.1 Abwehr

In unserer ersten Entwicklung eines Roboters gab es noch keine besonderen Arten der Abwehr. Sobald die Infrarotsensoren des Roboters keine Signale des Balls empfangen haben, zum Beispiel wenn der gegnerische Roboter mit dem Ball in eine Ecke gerichtet steht, haben wir ein so genanntes ‚Homing'-Programm ausgeführt. In diesem Programm ging es vor allem darum, mit Hilfe unserer Ultraschall-sensoren zurück vor unser eigenes Tor zu finden, um anschließend den Angriff des Gegners abzuwehren.

Bei der Entwicklung von zwei Robotern hat man jedoch mehrere Möglichkeiten, eine besonders gute Abwehr zu erlangen: Einer der beiden Roboter wird speziell nur für die Abwehr angepasst und programmiert, indem der Verteidiger nur mit einer Schussvorrichtung und der Angreifer mit einer Schussvorrichtung, sowie einem Dribbler ausgestattet sind. Die Aufgabe des Defenders (= Abwehrroboter) besteht darin, die Bälle vor dem Tor abzufangen und dem Attacker (= Angriffsroboter) bereitzustellen. Dies wird erreicht, indem der Defender sich während des Spiels meistens in der eigenen Hälfte aufhält und die Angriffe des Gegners verteidigt. Sobald der Defender dann den Ball in seinem Besitz hat, bringt er diesen in die Mitte des Spielfelds, um entweder selbst auf das Tor zu schießen oder dem Attacker den Ball zuzupassen. Es gibt einige Ausnahmen, die dem Defender seine Rolle erschweren, zum Beispiel, wenn es zum Fall des Einzelspielers wie in Kapitel 3.3.4 genauer erläutert, kommt. Dazu ist es wie bereits genannt wichtig, die weiße Außenlinie sowie die Linie des Torraums nicht vollständig zu überqueren, um eine Auszeit für den Defender zu vermeiden.

3.3.2 Angriff

Im 1vs1 Soccer (V1 Entwicklung) ist der Angriff recht einfach gehalten. Der Roboter versucht, den Ball zu finden und diesen unter Kontrolle zu bringen. Sobald dies gelungen ist, wird durch die Kamera die Torposition bestimmt und diese Position angefahren.

Falls sich der Ball in einer der beiden Ecken der gegnerischen Spielfeldhälfte befindet und der eigene Roboter so bei der Ballanfahrt durch die Kamera keine Sicht mehr auf das Tor hat, verwenden wir zunächst unsere Ultraschallsensoren, um die Position in der Ecke zu bestätigen und eine Einschränkung des Sichtfeldes durch einen gegnerischen Roboter auszuschließen.

Danach führen wir ein eigens für die Toranfahrt aus Ecken ohne Sicht auf das Tor angefertigtes Programm aus, bei dem wir uns so lange in Torrichtung (nach links bzw. rechts) bewegen, bis wieder Sichtkontakt hergestellt werden kann. Mit Hilfe dieses Programms ist es uns möglich, auch Bälle, die in Ecken liegen, zu verwerten.

Bei einem Spiel im 2vs2 Soccer (V2 Entwicklung) gibt es erschwerend eine Torauslinie, welche nicht überquert werden darf. Um zu verhindern, dass der Ball dauerhaft im nicht erreichbaren Bereich des Feldes liegen bleibt, wurde bei den Spielfeldern hinter der Linie eine kleine Erhöhung angefügt, um zu erreichen, dass der Ball wieder in das Feld rollt. Dies kann man sich im Angriff zunutze machen, da für einen Torversuch der Ball erst kontrolliert vor das Tor gebracht werden muss. So ist es möglich, durch gezieltes Anstoßen des Balls in eine Richtung, den Ball immer zwischen Wand und Roboter hin- und herzubewegen und ihn so aus dem Außenbereich zu entfernen.

Diese Taktik wird häufig von einem gegnerischen Roboter verhindert, welcher von hinten auf unseren Roboter drückt, um selbst den Ball zu erreichen. Um dies zu umgehen, versuchen wir mit dem zweiten Roboter die gegnerischen Roboter zu blockieren, bevor sie unseren Angreifer erreichen.

3.3.3 Anspiel

Bei einem offiziellen 1vs1 RoboCup Match (Entwicklungsstufe V1) wird zu Beginn des Spiels eine Münze geworfen. Das Team, das den Münzwurf für sich entscheidet, darf aussuchen, ob es ein Anspiel ausführen möchte oder auf der Seite als Verteidiger startet. Das zweite Team startet in der jeweils anderen Position.

Um ein Anspiel auszuführen wird der Roboter innerhalb des Mittelkreises auf der eigenen Hälfte so positioniert, dass er möglichst nahe am Ball liegt, diesen jedoch nicht berührt. Dies verschafft dem Team einen kleinen Zeitvorsprung im Erreichen des Balles und Ausführen des ersten Angriffes gegenüber dem Verteidiger des Roboters des anderen Teams, der in dem Moment als Verteidiger agiert. Der verteidigende Roboter darf dann an einer beliebigen Stelle innerhalb seiner Hälfe platziert werden. Wichtig ist allerdings, dass er die Linie des Mittelkreises weder berührt noch überschreitet und so einen Abstand von mindestens 30cm zum Ball einhält. Sobald der anspielende Roboter platziert wurde, ist eine Änderung der Position durch das Team nicht mehr erlaubt.

Im weiteren Spielverlauf ist immer das Team zum Anspiel - also der Positionierung innerhalb des Mittelkreises berechtigt, welches zuletzt ein Tor kassiert hat.

Da der Ball beim Anspiel direkt vor beiden Robotern liegt, ist ein sauberer Angriff, beziehungsweise eine saubere Verteidigung essenziell, um dem Gegner nicht direkt einen recht unkomplizierten Angriff zu ermöglichen. Verliert man direkt beim Anspiel die Kontrolle über den Ball, ist die Wahrscheinlichkeit groß, dass der gegnerische Roboter ein Tor erzielt, da die Spielsituation unübersichtlich ist.

Um ein erfolgreiches, kontrollverlustfreies Anspiel zu ermöglichen, haben wir mehrere verschiedene Taktiken ausprobiert und weiterentwickelt. Besonders bewährt hat sich eine Kurvenfahrt, direkt nach Erstkontakt zum Ball, die sowohl unseren Roboter als auch den Ball um den gegnerischen Roboter herummanövriert und uns so eine Torchance ermöglicht.

Ein weiterer Ansatz, der sich für die Position des Verteidigers, also der Positionierung des Roboters beim Anspiel außerhalb des Mittelkreises anbietet, ist eine rasche Fahrt nach hinten ins eigene Tor. Durch diese Aktion verhindern wir nicht nur ein Tor des Gegners, sondern verschaffen uns zusätzlich einen guten Überblick über das Spielfeld, da wir das gegnerische Tor mit der Kamera erfassen können und der Ball großflächig und zuverlässig von unseren IR-Sensoren geortet werden kann. Ein schneller Gegenangriff ist durch die frühzeitige Positionierung im Torraum und der Verteidigung möglich.

Neben diesen zwei Taktiken gibt es noch weitere Taktiken, wie eine vorteilhafte Positionierung des Roboters beim Anspiel je nach Verhalten des gegnerischen Roboters und Spielsituation erreicht werden kann

Auf ein 2vs2 Match (= Entwicklungsstufe V2) lassen sich fast alle unserer erarbeiteten Anspiel-Taktiken übertragen. Der jeweils zweite Roboter beider Teams wird in der eigenen Spielfeldhälfte außerhalb des Mittelkreises positioniert. In der Regel agiert einer der beiden Roboter als Angreifer, welcher sich zum

Beispiel durch das Fahren einer Kurve Kontrolle über den Ball verschafft, und der andere als Verteidiger, der zeitgleich den Torraum gegen einen eventuellen Gegenangriff absichert.

Bis jetzt haben wir noch keine finale Anspiel-Taktik entwickelt, was daran liegt, dass die zu verwendenden Taktiken situationsabhängig sind und oft erst auf dem Wettbewerb finalisiert werden können.

3.3.4 Einzelspieler

Im Entwicklungsstadium V2 kann es bei einem Regelverstoß eines Roboters, wie z.B. einem Überqueren einer Linie, zu einer 1-minütigen Zeitstrafe kommen. In diesem Fall kann sich der andere Roboter nicht darauf verlassen, einen Angriffs- bzw. Abwehrspieler bei sich zu haben. Deshalb muss eine solche Situation automatisch erkannt werden und eine sogenannte 'One-Player-Taktik' aktiviert werden, welche darauf ausgelegt ist, beide Rollen also Angriff und Abwehr, allein erfolgreich zu meistern.

4. Debuggen und Tools

Eines der wichtigsten und grundlegenden Themen, welche wir beachten mussten, war es, ein gutes Debugging-Verfahren während der Softwareentwicklung zu erreichen.

4.1 Erfolgreiche Datenerfassung

Das Erfassen von Messdaten ist sehr wichtig, um schnell eine funktionierende Software, welche effizient und schnell agiert, zu entwickeln. Vor allem, da diese auf jede erdenkliche Weise zu testen ist. Erfasste Messwerte sind jedoch erst dann interessant und hilfreich, wenn diese logisch und verständlich dargestellt werden können.

Angefangen haben wir mit einem einfachen Pythonscript, welches lediglich die geloggten Daten und Messwerte in eine für Excel lesbare CSV-Datei verpackte und wir diese mit Excel und später mit einem weiteren Python-Programm per Mathplotlib veranschaulicht haben. Damit hatten wir allerding eine sehr ungenaue und langsame Debugging-Methode, welche sehr aufwändig zu verwenden und fast unmöglich für schnelles Testen verfügbar war. Diese Methode konnten wir jedoch sehr gut in der normalen Arduino Umgebung nutzen, mit welcher wir zu Beginn unseres Projekts anfingen.

Nachdem wir inzwischen auf Platformio und der Software-Umgebung Visual Studio Code umgestiegen sind, entdeckten wir die Erweiterung ‚Serial Plotter' in VS-Code, mit der es sehr einfach möglich ist, sowohl fast unendlich viele Messdaten gleichzeitig in einem farbigen Graphen darzustellen als auch, diese als Zahlenwert mit minimal- / maximal- und Medianwert live zu verfolgen. Ein Speichern der Messdaten bzw. Verändern von Parametern ist hierbei jedoch nicht möglich.

Inzwischen verwenden wir eine Mischung aus der seriellen Methode und unserem neuen Network-Debugger (siehe Kapitel 5.3)

4.2 Bluetooth & Network Debugger

Um die seriellen Daten zur Datenerfassung an einen PC zu übertragen, haben wir schnell gemerkt, dass auf einem Spielfeld mit fast zwei Metern Länge und einem sehr schnellen und aggressiv fahrenden Roboter, ein einfaches USB-Kabel sowohl für die Mechanik der USB-Stecker, als auch für den Roboter selbst, schlecht und nicht wirklich praktikabel ist. Deshalb sind wir schnell dazu übergegangen, ein Bluetoothmodul zur Wireless-Datenerfassung zu verwenden. So ist es uns möglich, ganz ohne ein Kabel, serielle Messdaten zu empfangen und aufzuzeichnen.

Das Problem ist jedoch, dass dieses Verfahren nicht für den Upload von Programmen / Software auf den Main Controller (Teensy 4.0) geeignet ist. Auch ist eine Datenaufzeichnung per Bluetooth mit zwei Robotern und einem PC nur schwer zu meistern. Die Lösung ist ein eigenständiger Computer, welcher zu Testzwecken auf dem Roboter montiert wird, die Daten aufzeichnet und die Software flashed.

Für diesen Zweck verwenden wir einen Raspberry Pi Zero W, welcher sich in ein, vom PC bereitgestelltes Hotspot Netzwerk, einklinkt. Innerhalb dieses Netzwerkes ist es nun möglich, mit einer selbst entwickelten Python GUI[2] an den Raspberry eine Datei zu versenden. Sobald sich die Programmdatei auf dem Raspberry verändert, führt dieser einen Programmier-Vorgang durch und flashed den Microcontroller mit der neuen Software. Somit ist es möglich einen Teensy Wireless neu zu programmieren. Dieser Vorgang funktioniert sehr zuverlässig und benötigt nur einige Millisekunden. Durch den zentralen Hotspot auf dem PC ist es nun auch möglich, mehrere Roboter / Raspberrys auf einmal zu verwenden.

Genanter Raspberry Pi Zero W zum Wireless Programmieren und Testen des Roboters.

Um darüber hinaus eine Live-Datenerfassung und Parametereinstellung zu verwenden, ist die Kommunikation vom PC direkt zum Controller weiterhin unerlässlich. Hierfür sind wir erst kürzlich auf die Möglichkeit gestoßen in einem lokalen Netzwerk, UDP[3] Nachrichten zu versenden. Über solche UDP-Messages ist es nun möglich, die auf dem Raspberry per serielle Schnittstelle empfangen Daten, durch das Netzwerk an den PC weiterzuleiten und umgekehrt. Dieses Vorgehen nennen wir ‚Serial bridge‘, da der Raspberry eine direkte serielle Verbindung über das Netzwerk mit dem Controller simuliert.

4.3 Hardware-Interface

Eine große Rolle bei der Kommunikation zwischen Roboter und Anwender spielt das Interface, also die Knöpfe, die Anzeige LEDs, das Display und die Schalter. Vor allem ist es wichtig, dass die Bedienung des Roboters Intuitiv abläuft, da sonst im Stress des Wettbewerbs oft Fehler unterlaufen.

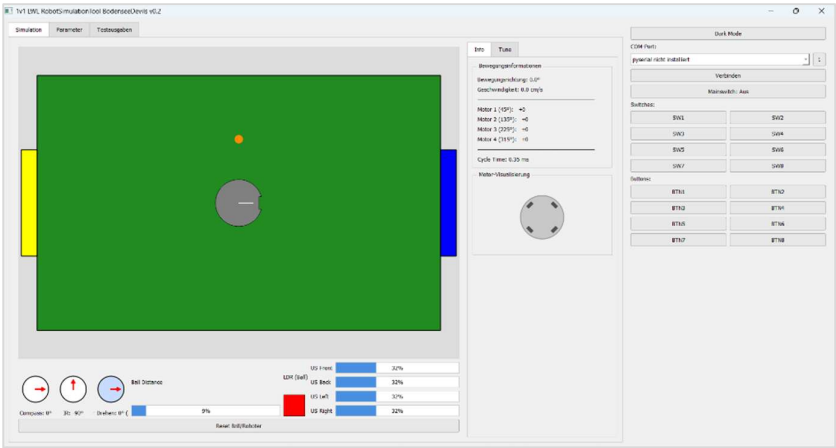
Um dies möglichst zu vermeiden, verwenden wir auf unserer Mainplatine insgesamt drei RGB LEDs so wie ein Display, welches zum Testen angesteckt werden kann, um direkt Messdaten anzuzeigen und Fehler schnell zu erkennen. Der Roboter verfügt zudem über drei Konfigurations-Schalter zum Verändern

[2] Grafikal User Interface
[3] User Datagram Protocol

von Spielzügen und Taktiken während des Spiels. Dies ist ein wichtiger Teil der Spiel-Logik und des Erfolges im Wettbewerb. Die Taktiken müssen immer wieder über das Spiel hinweg verändert werden, um dem Gegner nicht die Möglichkeit zu bieten, diese vorauszusagen.

5. Simulation

Aufgrund dessen, dass in den frühen Entwicklungsstadien jeder neuen Version kaum etwas 'Fahrbares' existiert, an welchem die Software getestet werden kann, haben wir uns mit einem anderen Team aus unserer AG zusammengesetzt und haben eine kleine Software-Umgebung programmiert, in der das gesamte Umfeld des Roboters simuliert werden kann.



Um die Kompatibilität des eigentlichen C++ Codes, welcher später auf dem Roboter verwendet wird, zu gewährleisten, nutzen wir dennoch einen standardisierten Teensy 4.0 Microcontroller zur Simulation. Der Controller allein wird an den PC angeschlossen und mit dem normalen Programm beschrieben. Dieses sendet jegliche Aktionen, wie Motorsteuerungen oder Anzeigen per LED an den Computer. Dieser bewegt den Roboter so in der virtuellen Umgebung, dass die Bewegungen mit den Befehlen des Kontrollers übereinstimmen. Zusätzlich errechnet der PC, welche Werte die einzelnen Sensoren in der jeweiligen Situation herausgeben würden, und teilt dies dem Controller mit.

So ist es möglich, ohne Abänderung der Software auf dem Teensy, das Spiel vollständig zu simulieren und somit die Software zu testen. Dies war in der Vergangenheit ein großer Vorteil für uns, da das Testen der Software oft nicht an einem der wöchentlichen Treffen in der Schule stattfinden kann, da hier vorrangig an der Hardware des Roboters gearbeitet wird.

6. Technische Herausforderungen

6.1 Entwicklung und Design der Platinen

Da wir nun in einer neuen Liga mit höheren Anforderungen teilnehmen, haben wir neue Platinen, wie zum Beispiel die Linienplatine, benötigt. Anfang der Sommerferien 2025 haben wir begonnen, uns mit dem Programm Eagle von Autodesk bekannt zu machen. Die Entwicklung der Platinen war eine der Zeit- sowie arbeitsintensivsten Vorgänge. Es gab immer wieder Verbesserungs- sowie Änderungsvorschläge von unseren Betreuern und Erkenntnisse im Team. Nach mehreren Monaten Entwicklung konnten wir die Arbeit an den neuen Platinen final abschließen und diese anschließend bestellen.

Daran anschließend stand das Bestücken dieser Platinen an. Dies benötigte sehr viel Geduld und Konzentration. Die erste Inbetriebnahme der Platine verlief glücklicherweise sehr gut und wir konnten bei weiteren Tests keine Fehler in der Platine mehr feststellen.

6.2 Entwicklung unter begrenztem Bauraum

Bereits in der Entwicklungstufe 1 (V1) war der Bauraum innerhalb des Roboters sehr begrenzt. Da die damals vorhandenen drei Motoren den Unterbau, beziehungsweise das Fahrwerk großflächig ausfüllten, war es schwierig einen geeigneten Platz für unseren Akku zu finden. Die Steifheit der dicken Akkukabel sorgte für eine weitere Einschränkung in der Positionierung.

Unser Dribbler Modul umfasste einen Motor, der ähnliche Maße wie die drei Fahrmotoren hatte, eine Walze, die den Kontakt zum Ball ermöglicht und eine Vorderwand. Um all dies in dem schmalen Unterbau unterzubringen, mussten wir mit Zahnrädern arbeiten und den Motor und die Walze hintereinander montieren. Möglich wurde dies durch eine aufwendige Halterung.

In der Entwicklungsstufe 2 besitzt jeder unserer Roboter, wie bereits erwähnt, vier statt drei Fahrmotoren. Der Bauraum wird also weiter eingeschränkt und macht eine Weiterentwicklung komplizierter und aufwendiger. Eine Positionierung des Akkus im Unterbau neben den Fahrmotoren wurde so unmöglich. Wir konnten dennoch nach der Konstruktion einer Halterung einen anderen geeigneten Platz für unseren Akku finden.

Als besonders schwierig gestaltete sich der Einbau des ‘Kicker-Moduls’. Es galt ein Modell zu finden, dass zugleich eine hohe Leistung, als auch eine kompakte Größe und Form hatte. Wir entschieden uns für eine quaderförmige Schussvorrichtung, welche wir durch eine Verbesserung der Wicklungen optimierten.

7. Quellen

https://robocup-junior.github.io/soccer-rules/master/rules.pdf

https://junior.robocup.de

https://markdorf-robotics.de